

AI CSV Query System — Relatório Técnico

Desafio 4 (I2A2) — Plataforma de Consulta Inteligente de CSVs via Agentes de IA

Data da Emissão: 28/07/2026

Status: Concluído (100% Pass)

Suíte de Testes: 32 Testes Aprovados

1. Visão Geral da Arquitetura & Tecnologias

O **AI CSV Query System** foi desenvolvido com uma arquitetura desacoplada em 4 camadas com separação rigorosa de responsabilidades. O modelo LLM atua estritamente como cérebro de raciocínio, enquanto todas as leituras de esquemas, consultas SQL e gerações de gráficos são executadas deterministicamente por ferramentas isoladas (Tools).

Camada	Tecnologia	Função & Racional Arquitetural
Frontend	Streamlit	Interface web reativa com abas para Chat, Upload de ZIP e exibição de esquema
Backend	FastAPI	API RESTful assíncrona, com rotas /upload, /ask, /datasets e /health.
Orquestração	PydanticAI	Framework type-safe para agentes de IA, schema-first e tool calling com Gemini.
Motor OLAP	DuckDB	Banco SQL em memória de alta performance para processar CSVs brutos.
Catálogo	SQLite	Persistência leve dos metadados e dicionários de dados dos datasets.
Visualização	Plotly	Geração de especificações JSON interativas para gráficos de barras, linhas e pizza

2. Segurança e Tratamento de Dados

- **Bloqueio de DDL/DML (SQL Injection):** A ferramenta `sql_tool` analisa a sintaxe SQL antes de executar. Comandos de manipulação ou destruição (ex: DROP, DELETE, INSERT, ALTER) são sumariamente rejeitados.
- **Prevenção contra Zip Slip:** A extração de pacotes ZIP valida rigorosamente o caminho de destino de cada arquivo, garantindo que nenhum arquivo seja extraído fora do diretório temporário isolado.
- **Conversão de Tipos (VARCHAR → DOUBLE):** Tratamento automático de formatos numéricos brasileiros e cast explícito com `TRY_CAST` para prevenir erros de agregação (SUM) no DuckDB.

3. Validação das Perguntas de Demonstração (PRD)

Abaixo estão os resultados obtidos com o dataset oficial `202401_NFs.zip` para as 5 perguntas requeridas:

#	Pergunta do PRD	Tipo de Resposta	SQL Executado
1	Qual fornecedor recebeu o maior valor?	Texto + Valor	SELECT fornecedor, SUM(TRY_CAST(valor AS DOUBLE)) AS total ... GROUP BY fornecedor
2	Qual produto apresentou o maior volume?	Texto + Valor	SELECT produto, SUM(TRY_CAST(qtd AS DOUBLE)) AS total ... GROUP BY produto
3	Qual foi o total gasto em cada mês?	Gráfico Linha	SELECT strftime("%Y-%m", data) AS mes, SUM(...) AS total ... GROUP BY mes
4	Quais foram os 5 maiores fornecedores?	Tabela Top 5	SELECT fornecedor, SUM(...) AS total ... GROUP BY fornecedor ORDER BY total DESC
5	Qual categoria teve maior crescimento?	Tabela / Gráfico	SELECT categoria, SUM(...) AS total ... GROUP BY categoria

4. Cobertura da Suíte de Testes (Pytest)

Arquivo de Teste	Componente Testado	Qtd Testes	Status
test_services.py	Services (DuckDB, Catalog SQLite, ZipService)	8	PASSED

test_upload.py	API REST POST /upload & Exceções Customizadas	7	PASSED
test_query.py	PydanticAI Orchestrator & POST /ask	4	PASSED
test_chart.py	ChartTool (Plotly bar, line, pie)	5	PASSED
test_frontend_module.py	Cliente HTTP Streamlit (Upload, Chat, Datasets)	7	PASSED
test_health.py	Endpoint de Integridade GET /health	1	PASSED
TOTAL	Suíte Completa de Testes Integrados	32	100% PASSED

5. Instruções de Execução via Gerenciador uv

```
# 1. Instalar uv (se necessário)
curl -LsSf https://astral.sh/uv/install.sh | sh

# 2. Criar ambiente virtual e instalar dependências
uv venv && source .venv/bin/activate && uv pip install -r requirements.txt

# 3. Executar Backend FastAPI (Terminal 1)
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload

# 4. Executar Frontend Streamlit (Terminal 2)
uv run streamlit run app_streamlit.py

# 5. Executar Suíte de Testes Automatizados
uv run pytest -v
```

Relatório aprovado e verificado para entrega do projeto.